Team Name: Lions
Team Members: Iesus Gonzalez-Flores, Chris Salcedo Cardenas, Edgar Pena Arreola
CS 2430-501 Discrete Structures Spring 2026
Programming Project 1

Introduction: To facilitate user access to services such as databases or online stores, it helps to allow users to search and ask questions in natural language. That's where a Natural Language Query (NLQ) comes in. A natural language query allows a user to find what they need by simply asking a question without needing to know the intricate workings of the software they are interfacing with. One example of an NLQ can be a chatbot on a website. For example, if a user was shopping online and wanted to find "a pink dress with black polka dots", the user shouldn't need to worry themselves with writing instructions to access that online store's inventory to purchase the item. The issue, however, is that natural language often contains a fair amount of ambiguity. For example, if a user was shopping online and asked the question, "I need 4 large house plants," the ambiguity would lie in defining what a large house plant would be. The NLQ would need to have some kind of categorization for small, medium, and large house plants to be able to return results for "large house plants."

For our project, we decided to create sets and multisets to keep track of animals in a zoo. We provided functionality of basic set operations such as union, intersection, difference, symmetric difference, and complement. Our set's implementation is capable of organizing animals in our zoo alphabetically, and our multiset implementation is capable of keeping a count of how many of each kind of animal can be found in our zoo.

Implementations: For our ordinary sets, we used the TreeSet data structure. The TreeSet data structure allows our data to remain organized in alphabetical order. One huge benefit that TreeSet provides is that we can continue to grow our universal set and not have to worry about keeping the list of animals in our zoo organized. Like any other set, TreeSet doesn't keep duplicates. We further improved this characteristic of TreeSet by assuring that items were made lowercase before being added to our TreeSet to ensure that our list of zoo animals didn't contain duplicates. For our multisets, we chose to use the HashMap data structure for its key-value mapping. Each animal in our zoo would act as its own special key, and the count of each animal would be the associated value. Therefore, HaspMap allows a user to retrieve an animal from our zoo, and with that animal comes its associated mapped value, in this case, the count of that animal in our zoo. Using HashMap means we don't need to create separate counts for each animal in our zoo.

Verification Approach: For our ordinary sets, we tested basic set operations, such as union, by creating two subsets of our universal set of zoo animals and adding animals to each subset. We then combined both sets and returned a list of the new subset. We compared that subset to an

expected list of animals that should be present by performing the union set operation. For our multisets, we tested the intersection operation by having two multisets of animals whose animal count field implemented HashMap. First, we compared two multisets, multiset Zoo1 and multiset Zoo2. We checked if the key, or in this case, the animals of multiset Zoo2, was found in multiset Zoo1. If a matching animal was found, we compared the count of the animals and kept the smallest of the two. We then returned a new multiset that contained all animals of multiset Zoo1 and multiset Zoo2, and made the animal count of each animal the minimum of the two counts from multiset Zoo1 and multiset Zoo2. We finally compared the new multiset of zoo animals to a multiset of animals with an expected list of animals and animal counts. One edge case we tested for with our multisets was assuring that we weren't trying to perform set operations with null multisets. If a user tried to perform a set operation with null sets, they would receive an error stating that "Multisets cannot be null".

Results: Please note that the order of the following Bit String representations was based on the alphabetical universal set of animals below

```
private static final List<String> UNIVERSAL_ZOO = Arrays.asList("elephant", "giraffe", "gorilla",
    "hippo", "kangaroo", "lion", "penguin", "rhino", "tiger", "zebra");
```

The sets being operated on are Set A = {"lion", "elephant", "penguin", "kangaroo"} and
Set B = {"tiger", "elephant", "gorilla", "rhino" }

A ∪ B Union Set Operation Output
TESTING UNION OPERATION ---------- Expected: [elephant, gorrila, kangaroo, lion, penguin, rhino, tiger] Actual: [elephant, gorilla, kangaroo, lion, penguin, rhino, tiger] Bit String Representation: 1010111110

A ∩ B Intersection Set Operation Output

```
TESTING INTERSECTION OPERATION
------------------------------

Expected: [elephant]
Actual: [elephant]
Bit String Representation: 1000000000
```

A - B Difference Set Operation Output

```
TESTING DIFFERENCE OPERATION
----------------------------

Expected: [Kangaroo, Lion, penguin]
Actual: [kangaroo, lion, penguin]
Bit String Representation: 0000111000
```

A ⊕ B Symmetric Difference Set Operation Output

```
TESTING SYMMENTRIC DIFFERENCE OPERATION
---------------------------------------

Expected: [gorilla, kangaroo, lion, penguin, rhino, tiger]
Actual: [gorilla, kangaroo, lion, penguin, rhino, tiger]
Bit String Representation: 0010111110
```

NOT(A) Complement Set Operation Output

```
TESTING COMPLEMENT OPERATION
----------------------------

Expected: [giraffe, gorilla, hippo, rhino, tiger, zebra]
Actual: [giraffe, gorilla, hippo, rhino, tiger, zebra]
Bit String Representation: 0111000111
```

Please note that for the multiset Bit String representations, the universal set was the following:

```
UNIVERSAL SET (Zoo Animals)
U = [Lion, Tiger, Giraffe, Elephant, Zebra, Penguin, Gorilla, Hippo, Rhino, Kangaroo]
```

A ∪ B Union Multiset Operation Output

```
A ∪ B bits:  1101011011
A ∪ B elems: [Lion, Tiger, Elephant, Penguin, Gorilla, Rhino, Kangaroo]
```

A ∩ B Intersection Multiset Operation Output

```
A ∩ B bits:  0001000000
A ∩ B elems: [Elephant]
```

A - B Difference Multiset Operation Output

```
A \ B bits:  1000010001
A \ B elems: [Lion, Penguin, Kangaroo]
```

A ⊕ B Symmetric Difference Multiset Operation Output

```
A Δ B bits:  1100011011
A Δ B elems: [Lion, Tiger, Penguin, Gorilla, Rhino, Kangaroo]
```

NOT(A) Complement Multiset Operation Output

```
~A bits:  0110101110
~A elems: [Tiger, Giraffe, Zebra, Gorilla, Hippo, Rhino]
```

Testing Count Functionality of Multisets Output

```
================================
MULTISET TEST CASE
================================
Zoo1 counts: { Lion:3, Tiger:0, Giraffe:0, Elephant:2, Zebra:5, Penguin:10, Gorilla:0, Hippo:0, Rhino:0, Kangaroo:0 }
Zoo2 counts: { Lion:1, Tiger:0, Giraffe:4, Elephant:0, Zebra:2, Penguin:6, Gorilla:0, Hippo:0, Rhino:0, Kangaroo:0 }

Union (max): { Lion:3, Tiger:0, Giraffe:4, Elephant:2, Zebra:5, Penguin:10, Gorilla:0, Hippo:0, Rhino:0, Kangaroo:0 }
Intersection (min): { Lion:1, Tiger:0, Giraffe:0, Elephant:0, Zebra:2, Penguin:6, Gorilla:0, Hippo:0, Rhino:0, Kangaroo:0 }
Difference (zoo1 - zoo2): { Lion:2, Tiger:0, Giraffe:0, Elephant:2, Zebra:3, Penguin:4, Gorilla:0, Hippo:0, Rhino:0, Kangaroo:0 }
Sum (add counts): { Lion:4, Tiger:0, Giraffe:4, Elephant:2, Zebra:7, Penguin:16, Gorilla:0, Hippo:0, Rhino:0, Kangaroo:0 }
```

Discussion:

1.) It would be possible to have overloaded methods that provide the correct functionality regardless of using sets or multisets. For example, with our basic sets, we could have implemented overloaded methods. One method could have accepted two TreeSets and returned a TreeSet that contained the intersection of two sets, if we required a set to have its elements in a specific order. Another overloaded method could have accepted two HashMaps, utilizing the key-value mapping ability. Depending on whether a user passed two TreeSets or HashMaps, we could infer if they needed sets or multisets.

2.) It would be possible to use the same data structure to store both sets and multisets. We could utilize the HashMap data structure for both sets and multisets. If we need basic set operations, each key could simply have an associated value of "1," which we could disregard if we didn't need a count of the animals but rather a list of all animals in our zoo. If the count of animals mattered, we would then take advantage of the key-value mapping that HashMap provides.

3.) Determining whether we need to use a set or a multiset based on the user's natural language query can be very difficult. If a user doesn't specify in their question that they care about frequency, they may end up working with a set that doesn't provide the count of items in a multiset. We would have to look for keywords that provide hints as to what kind of set a user needs, such as "how many", "count", "total", "frequency" etc for multisets, and "unique", "distinct", "types of " keywords could lead us to infer a user wants to use a basic set.
4.) While it is possible to store data from a plain set in a multiset, we wouldn't be able to do the reverse without losing information that holds the count data. Sets only hold unique elements in them; they don't store the count of each element the way multisets do. So if we tried moving a multiset to a set, we would have the same elements but would lose information about how many of each element were present in the multiset. Concerning fuzzy sets, while they may help us with retrieving data from our sets when a user asks a question with ambiguity, they wouldn't prevent the loss of data from moving a multiset to a plain set.
5.) If a designer were trying to code an interface that would allow users to type in natural language queries, they could create overloaded methods that could help in determining whether a user wants a set or a multiset. Due to the ambiguity of human language, the designer would have to implement fuzzy sets that help with that ambiguity. If a user asks a vague question about a set, such as "Give me all tall people from this set," the designer would have implement fuzzy sets that characterize which person in the set is considered "tall" within a certain degree of membership.
6.) If a designer were trying to code an interface that would allow users to access arbitrary types of data using natural language, the designer's code would have to account for what kind of data type a user wants to work with. The user may also be trying to access various kinds of data. For example, their query may first include keywords like "show all items," which may indicate a display of a set, but if they follow the query with a new one, such as "how much of x do we have?", it should indicate to the designer that the user now wants to work with multisets.

Resources

"What is Natural Language Query (NLQ)?" *TechTarget*, www.techtarget.com/whatis/definition/natural-language-query. Accessed 4 Mar. 2026.